In [2]:
```
import pandas as pd
import numpy as np

import folium
from folium import plugins

from IPython.display import HTML
from IPython.display import IFrame
```

Folium

Now are are going to move on to folium and do very similar stuff but folium is for interactive mapping where you can place the map into a website or mail it to a friend. Again, remember mapping is buggy. But we will take our time and work through it. Here is a good site for an overview

https://blog.dominodatalab.com/creating-interactive-crime-maps-with-folium/

USE LOCAL PATHS!

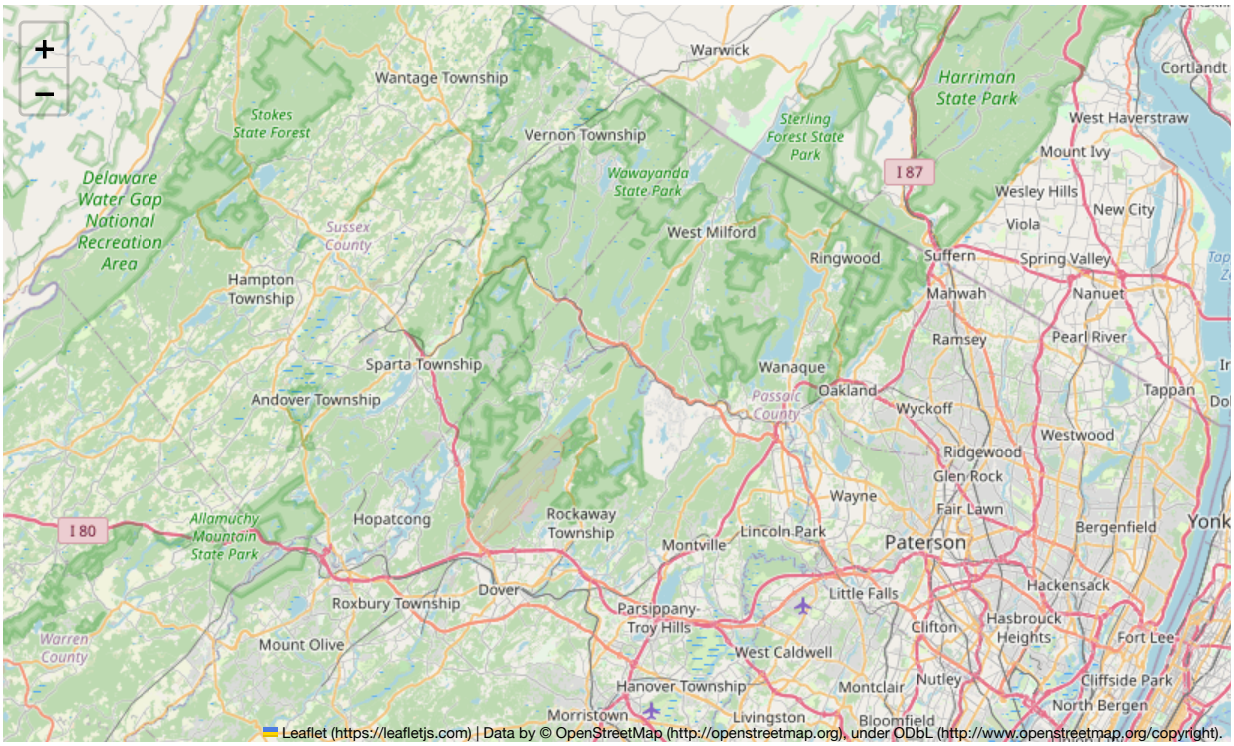
Here is a website I made and hosting it on github https://bmaillou.github.io/RedHookLead/

The first basic parts are

1. A location
2. Then you make a folium map
3. Then you display the map.

In [3]:
```
location =[40.8106,-73.9630]
m = folium.Map(location=location)
m
```

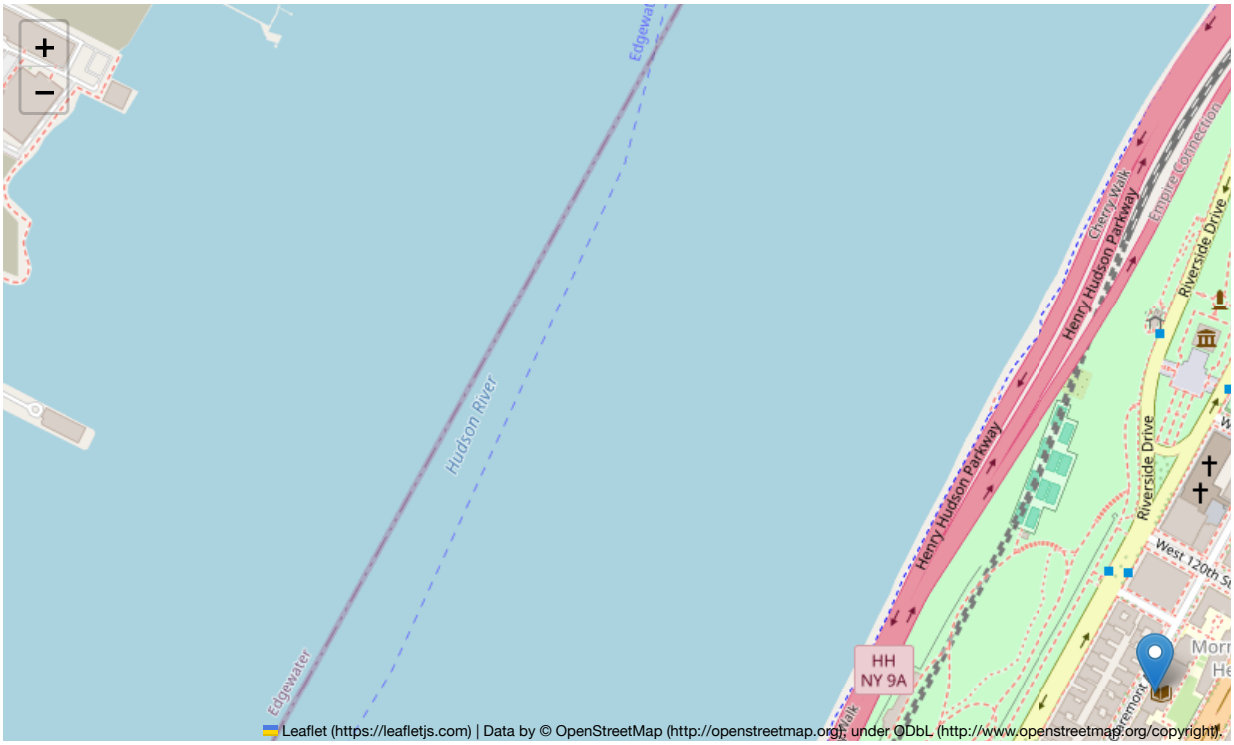
Out[3]:



Now we will add a popup with our room!

In [12]:
```
location =[40.8096,-73.9638]
m = folium.Map(location=location,zoom_start=16)
folium.Marker(location,popup='Wow This is our Room!').add_to(m)
m
```

Out[12]:



Amazing.

We put a point on our classroom and if you click the point it has a message! I want to make the maps smaller. The best way I did this was I save the map as an html file and then open the file with iframe. But they don't print! so I will comment out the lines and you can use them. It is weird.

In [15]:
```
mapName='FirstMap.html'

location =[40.8096,-73.9638]
m = folium.Map(location=location,zoom_start=16)
folium.Marker(location,popup='Wow This is our Room!').add_to(m)

#m  #This should show the map if you comment out the next two lines
m.save(outfile=mapName)  #saves to a file you can open
IFrame(mapName, width=700, height=300)  #opens in notebook
```

Out[15]:

Now go and find your html file and open it.

Let's look at some things we can change. We will go back to zoom_start=6 and try a few things.

First we can change what the map looks like!

https://deparkes.co.uk/2016/06/10/folium-map-tiles/

Here are some example tiles.

If we do help(folium.Map) we will learn a lot more.

So lets see what happens if we add the keywork tile='cartodb positron'

They have a few other options but I stick with the base

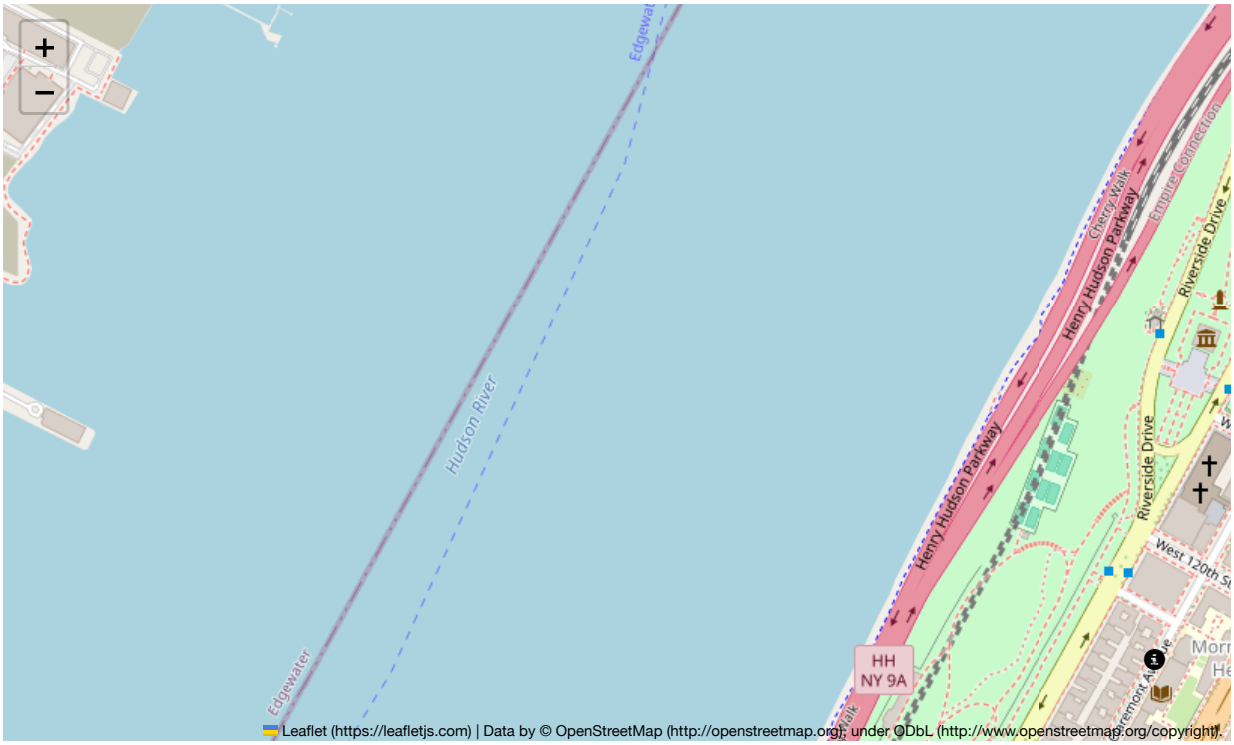
In [18]:
```
mapName='FirstMap.html'
location =[40.8096,-73.9638]
m = folium.Map(location=location,zoom_start=14,tiles="cartodb positron")
folium.Marker(location,popup='Wow This is our Room!',color='red').add_to(m)
m
```

Out[18]:

You can change the icon and add symbols to it. The quickstart https://python-visualization.github.io/folium/latest/getting_started.htmland the help will give you more information.

In [20]:
```
location =[40.8096,-73.9638]
icon=folium.Icon(color='red',icon='info-sign',icon_color='blue')
m = folium.Map(location=location,zoom_start=16)
folium.Marker(location,popup='Wow This is our Room!',icon=icon).add_to(m)
m
```

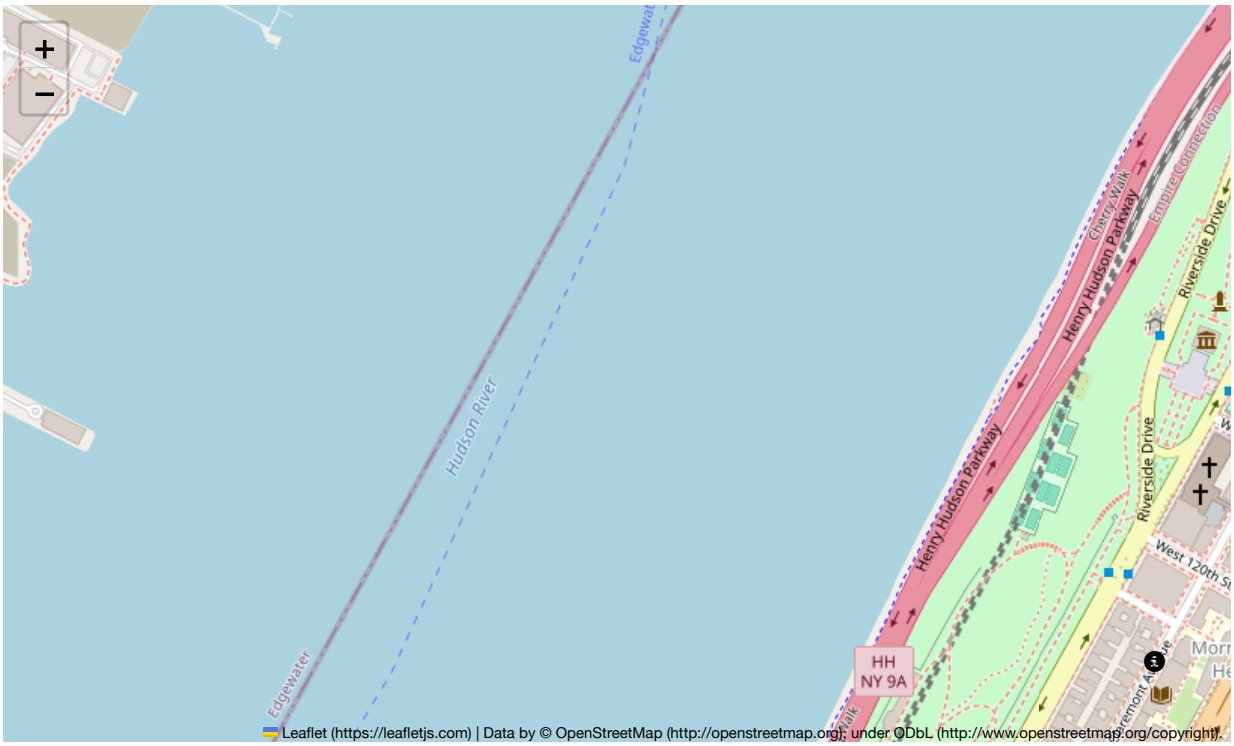
Out[20]:



In [ ]:

In [21]:
```
location =[40.8096,-73.9638]
icon=folium.Icon(color='red',icon_color='red')
m = folium.Map(location=location,zoom_start=16)
folium.Marker(location,popup='Wow This is our Room!',icon=icon).add_to(m)
m
```

Out[21]:



Now we are going to make an interactive map of lead in Brooklyn!

- First step is we need a circle marker

In [11]:
```
#help(folium.CircleMarker)
```

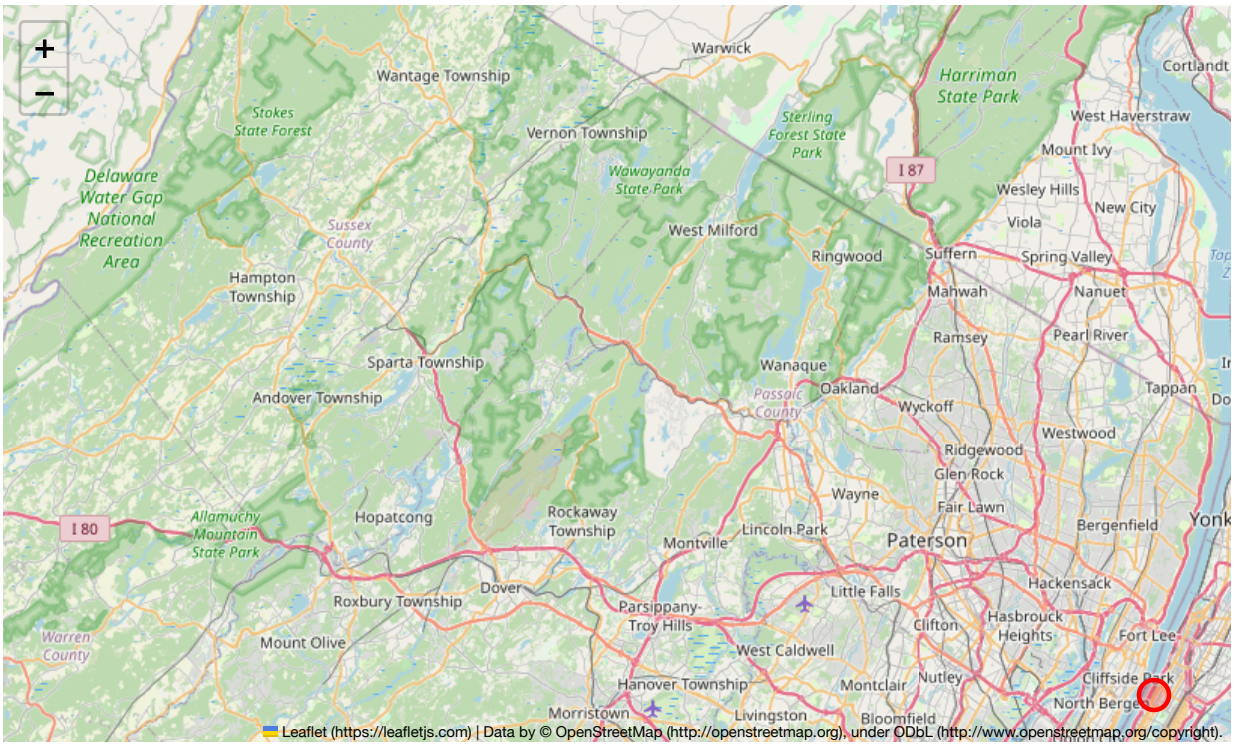
It looks like we need to pass

1. location
2. radius
3. fill_color
4. color
5. fill_opacity
6. popup

Lets try changing our icon to a circle

In [22]:
```
location =[40.8096,-73.9638]
m = folium.Map(location=location,zoom_start=10)
folium.CircleMarker(location,radius=10,popup='Wow This is our Room!'
                    ,color='red',fill_color='red').add_to(m)
m
```

Out[22]:



So let's read in the lead data

In [23]:
```
df=pd.read_excel('BrooklynPublicLead.xlsx',sheet_name='PublicData')
```

In [24]:
```
#df
```

Out[24]:

	SampleNum	sample_id	SubmissionDate	Latitude	Longitude	sample_gps.Altitude	descrip_other	Pb
0	265	17BP9001	2017-06-21 22:56:00	40.716585	-73.955596	-19.970294	NaN	122.333333
1	266	17BP9002	2017-06-21 18:05:00	40.716212	-73.954560	-67.265589	Near active construction site/ might be a school	85.666667
2	267	17BP9003	2017-06-21 18:05:00	40.716245	-73.954234	-28.343367	NaN	94.000000
3	268	17BP9004	2017-06-21 18:05:00	40.716487	-73.954213	-4.880463	Near garage	46.333333
4	269	17BP9005	2017-06-21 18:05:00	40.716542	-73.955192	-16.867952	NaN	1414.333333
...	...	...	...	...	...	...	...	...
458	723	17BP9478	2017-09-05 10:54:00	40.723305	-73.942755	15.000000	On Driggs near Henry. Other side of street fro...	51.666667
459	724	17BP9479	2017-09-01 15:40:00	40.722573	-73.943051	3.294742	On N Henry between driggs and engert	195.333333
460	725	17BP9480	2017-09-05 10:54:00	40.724796	-73.944247	4.000000	On Russell, between Driggs and Nassau. In fron...	528.666667
461	726	17BP9481	2017-09-05 10:54:00	40.726944	-73.944729	0.000000	On Russell near norman	477.000000
462	727	17BP9999 - A1 McCarren Park Reuters	2017-10-03 17:14:00	40.723151	-73.951878	0.000000	Sample taken on 9/14/17 after an insitu XRF me...	1037.500000

463 rows × 8 columns

This is where folium is a pain. You CAN NOT pass an array to CircleMarker. You can only pass one item at a time. So you need to for loop over each row. iterrows!

- make an iterrows for loop
- set the location
- check the lead and set the color. since we are going one at a time set the color with an if statement
- Add the name to the popup
- use your .format notation to make a name for the popup that is the sample id and lead concentration
- test the for loop and then add it to the map and call circel maker

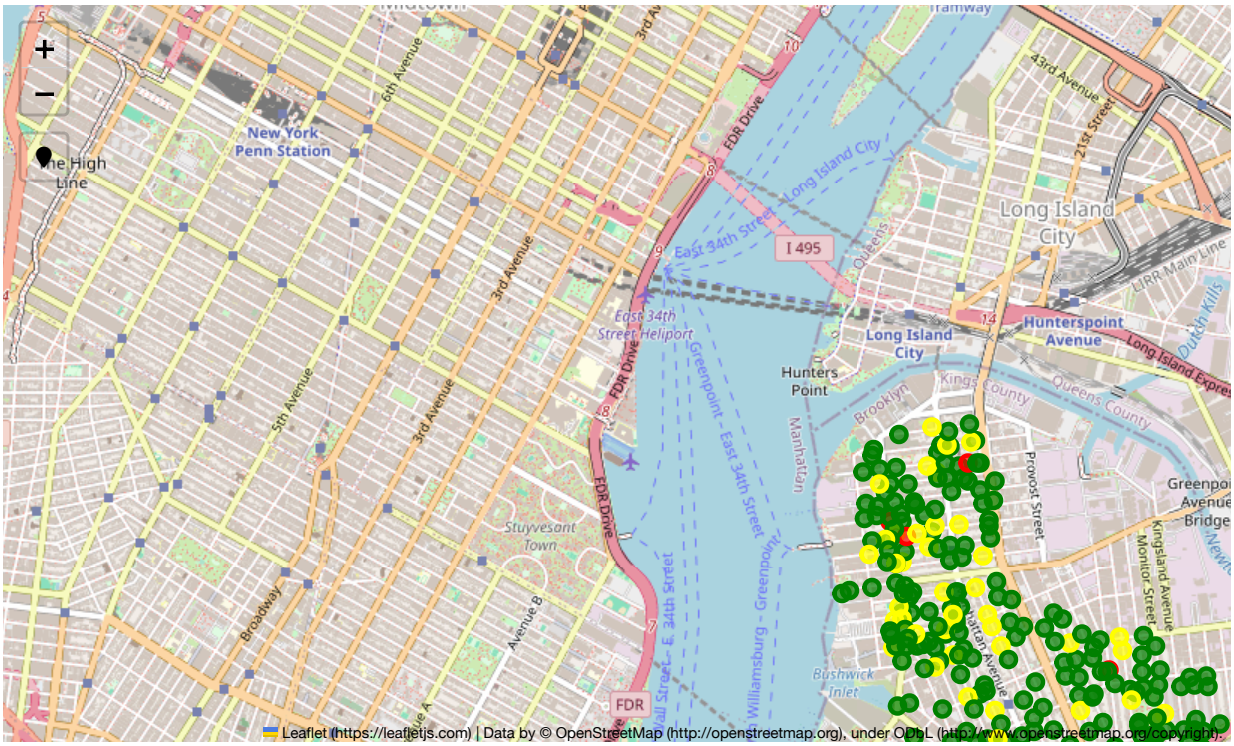
In [26]:
```
for idx, dfR in df.iterrows():
    #print('for index {} the lead is {} ppm'.format(idx,dfR['Pb']))
    # uncomment to run
```

```
  Cell In[26], line 3
    # uncomment to run
                      ^
SyntaxError: unexpected EOF while parsing
```

In [27]:
```
location =[40.725,-73.9430]
m = folium.Map(location=location,zoom_start=14)

for idx, dfR in df.iterrows():
    if dfR['Pb']<400:
        color='green'
    elif dfR['Pb']<1200:
        color='yellow'
```

```
    elif dfR['Pb']>=1200:
        color='red'
    else:
        color='blue'

    location =[dfR['Latitude'],dfR['Longitude']]
    text='id: {}\nPb: {:0.2f} mg/kg'.format(dfR['sample_id'],dfR['Pb'])
    folium.CircleMarker(location=location,radius=5,popup=text
                        ,fill=True,color=color
                        ,fill_color=color,fill_opacity=0.7).add_to(m)

plugins.LocateControl().add_to(m)
mapName='BrooklynLead.html'
m.save(outfile=mapName)  #saves to a file you can open

m
```

Out[27]:



Now you have an html file you can share!

The html is on your computer and you can turn it into a website or share

Some Fun extras

- adding photos
- coloring states

Someone asked about adding photos to the popup. here it is. This is just for reference. Took me a while to figure it out.

- I add a column in my excels saying where I store the photo on my computer
- the photos need to be small in size
- you then read in the photo. convert it. add it to the text. then add it to the popup.

In [28]:
```
import base64
```

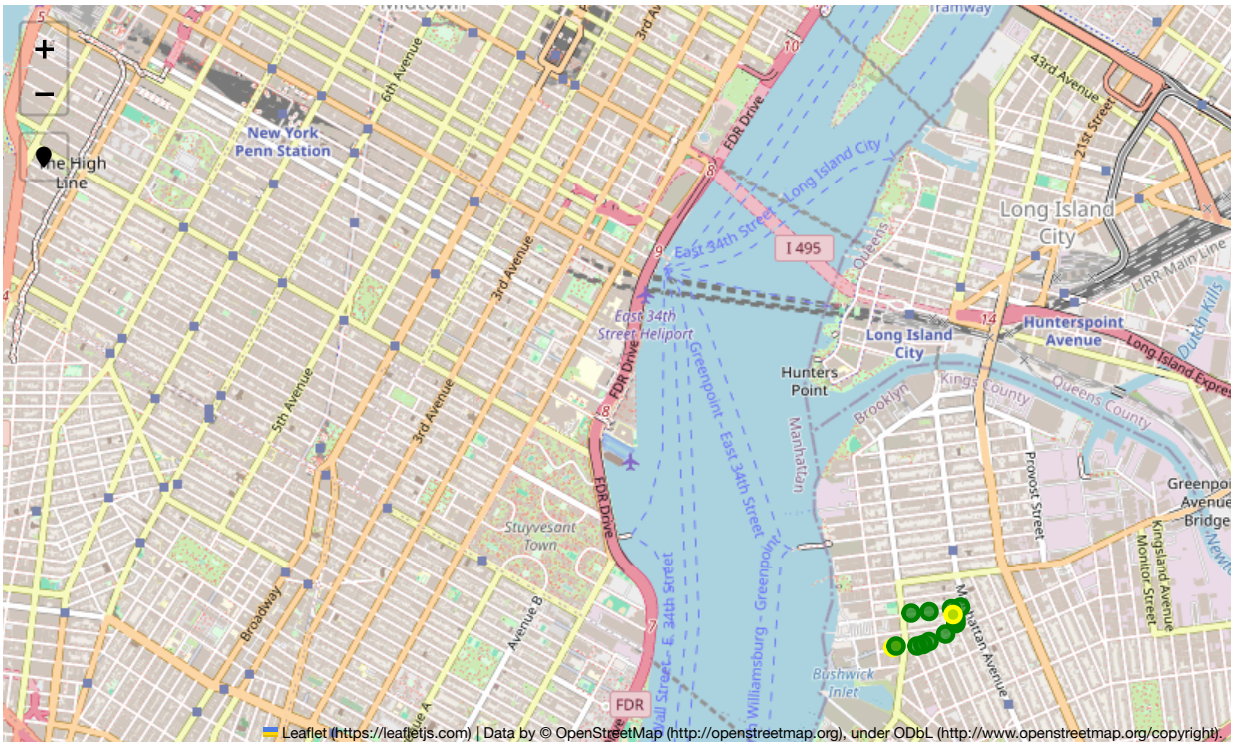
In [29]:
```
df=pd.read_excel('BrooklynPublicLead.xlsx',sheet_name='withPhotos')
```

In [41]:
```
#df
```

In [ ]:

In [40]:
```
location =[40.725,-73.9430]
m = folium.Map(location=location,zoom_start=14)

for idx, dfR in df.iterrows():
    if dfR['Pb']<400:
        color='green'
    elif dfR['Pb']<1200:
        color='yellow'
    elif dfR['Pb']>=1200:
        color='red'
    else:
        color='blue'

    location =[dfR['Latitude'],dfR['Longitude']]
    text='id: {}\nPb: {:0.2f} mg/kg'.format(dfR['sample_id'],dfR['Pb'])

    ##### This adds the photo
    filename=dfR['photo']
    data_uri = base64.b64encode(open(filename, 'rb').read()).decode()
    html = '<img src="{0}">'.format(data_uri)
    text=text+html
    iframe = folium.IFrame(text,
                    width=100,
                    height=400)


    folium.CircleMarker(location=location,radius=5,popup=text
                        ,fill=True,color=color,fill_color=color,fill_opacity=0.7).add_to(m)

plugins.LocateControl().add_to(m)
mapName='BrooklynLead.html'
m.save(outfile=mapName)  #saves to a file you can open

m
```

Out[40]:



The next cell just colors by state. Just here as more extra

In [52]:
```
state_geo = r'cb_2016_us_state_5m/cb_2016_us_state_5m.geojson'

m = folium.Map(location=[48, -102], zoom_start=3)
```

```
m.choropleth(
    geo_data=state_geo
)
folium.LayerControl().add_to(m)

#m
m.save(outfile='states.html')

IFrame('states.html', width=700, height=350)
```

```
/Users/bmaillou/anaconda3/envs/BigDataPython2025/lib/python3.8/site-packages/folium/folium.py:465: FutureWa
rning: The choropleth  method has been deprecated. Instead use the new Choropleth class, which has the same
arguments. See the example notebook 'GeoJSON_and_choropleth' for how to do this.
  warnings.warn(
```

Out[52]:

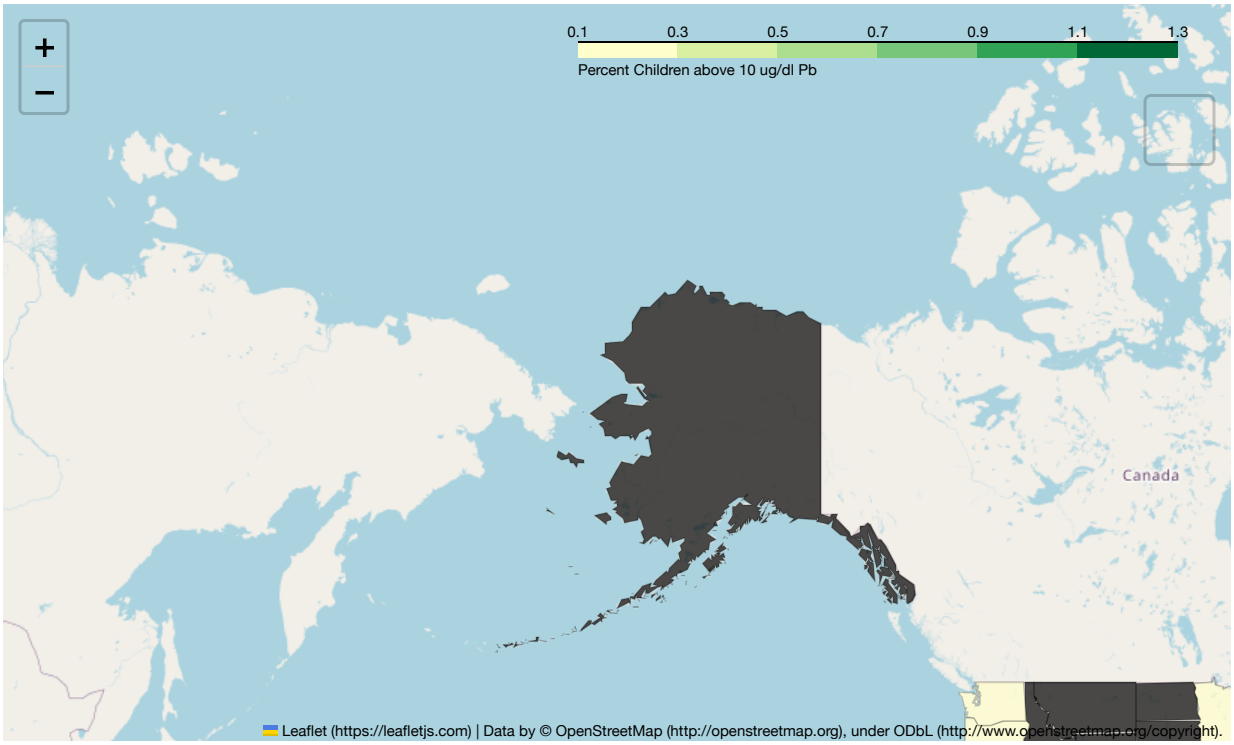
In [44]:
```
dfPb=pd.read_excel('BloodLeadDataStateFolium.xlsx')
```

In [46]:
```
dfPb.columns
```

Out[46]:
```
Index(['State', 'StateAbbrev', 'Population_less_than_72_months',
       'NumberChildrenTested', 'TotalChildrenBLL', 'Perct10dl'],
      dtype='object')
```

In [70]:
```
#"Feature", "properties"

m = folium.Map(location=[48, -102], zoom_start=3)
state_geo = r'cb_2016_us_state_5m/cb_2016_us_state_5m.geojson'

folium.Choropleth(
    geo_data=state_geo,
    name="choropleth",
    data=dfPb,
    columns=["State", "Perct10dl"],
    key_on="feature.properties.NAME",
    fill_color="YlGn",
    fill_opacity=0.7,
    line_opacity=0.2,
    legend_name="Percent Children above 10 ug/dl Pb",
).add_to(m)

folium.LayerControl().add_to(m)

m
```

Out[70]:



In [ ]:

In [ ]:

In [ ]:

In [ ]: